

Visualization (ggplot2, matplotlib, & seaborn)

Lecture 11

Dr. Colin Rundel

Grammar of graphics

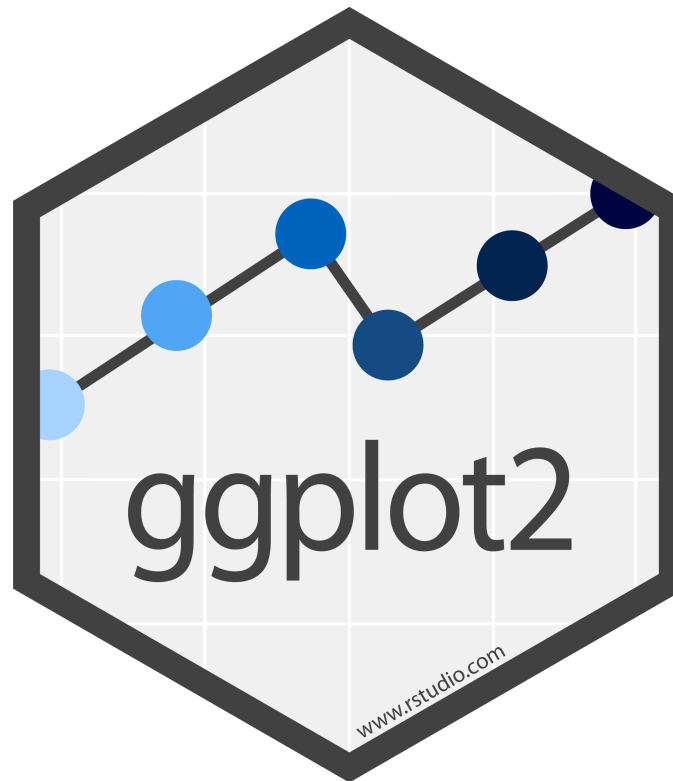
The Grammar of Graphics

- Conceptualized by Leland Wilkinson in The Grammar of Graphics (1999)
- An attempt to taxonomize the basic elements of statistical graphics - what a plot is made of, independent of any particular chart type
- Adapted for R by Hadley Wickham as ggplot2 (2007)
 - a consistent and compact syntax for describing statistical graphics
 - highly modular - a plot is assembled from semantic components rather than picked from a menu of chart types
 - opinionated about how a plot is described, not about which plot to make
- The same ideas underlie plotnine (a port of ggplot2 to Python) and seaborn's *objects* interface (end of the lecture)

Components of a plot

A statistical graphic is a mapping of data variables to aesthetic attributes (position, color, size, shape, ...) of geometric objects (points, lines, bars, ...), possibly after a statistical transformation, drawn in a coordinate system and possibly split into facets.

Component	Role	ggplot2
data	the data frame being plotted	<code>ggplot(data)</code>
aesthetics	which variables map to which visual properties	<code>aes()</code>
geometries	the visual objects that represent the data	<code>geom_*()</code>
statistics	transformations applied before drawing (counts, bins, model fits)	<code>stat_*()</code>
scales	how data values become aesthetic values (palettes, breaks, log)	<code>scale_*()</code>
facets	splitting the data into small multiples	<code>facet_*()</code>
coordinates	the coordinate system (Cartesian, flipped, polar, map)	<code>coord_*()</code>
theme	everything that is not data (fonts, gridlines, backgrounds)	<code>theme_*()</code> , <code>theme()</code>



Layers

A ggplot2 plot is a stack of layers drawn on shared scales, facets, and coordinates, with a theme on top. Each layer combines data and an aesthetic mapping with a geom, a stat, and a position adjustment.

```
1 ggplot(  
2   data = [data frame],  
3   mapping = aes(x = , y = , ...)  
4 ) +  
5   geom_[type](  
6     aes(...), stat = , position =  
7   ) +  
8   geom_[type](...) +  
9   scale_[aes]_[type]() +  
10  facet_[type]() +  
11  coord_[type]() +  
12  theme_[name]()
```

R

Components are combined with `+`, not a pipe (ggplot2 predates both `%>%` and `|>`). A pipe would pass the plot as the

Data - Palmer penguins

```
1 library(palmerpenguins)
2 (penguins = tidyr::drop_na(penguins))
```

R

A tibble: 333 × 8

	species	island	bill_length_mm	bill_depth_mm
	<fct>	<fct>	<dbl>	<dbl>
1	Adelie	Torgersen	39.1	18.7
2	Adelie	Torgersen	39.5	17.4
3	Adelie	Torgersen	40.3	18
4	Adelie	Torgersen	36.7	19.3
5	Adelie	Torgersen	39.3	20.6

i 328 more rows

i 2 more variables: sex <fct>, year <int>

```
1 penguins = pd.read_csv(
2     "data/penguins.csv"
3 ).dropna()
4 penguins
```

py

	species	island	bill_length_mm	...
0	Adelie	Torgersen	39.1	...
1	Adelie	Torgersen	39.5	...
..	...	...	...	...
342	Chinstrap	Dream	50.8	...
343	Chinstrap	Dream	50.2	...

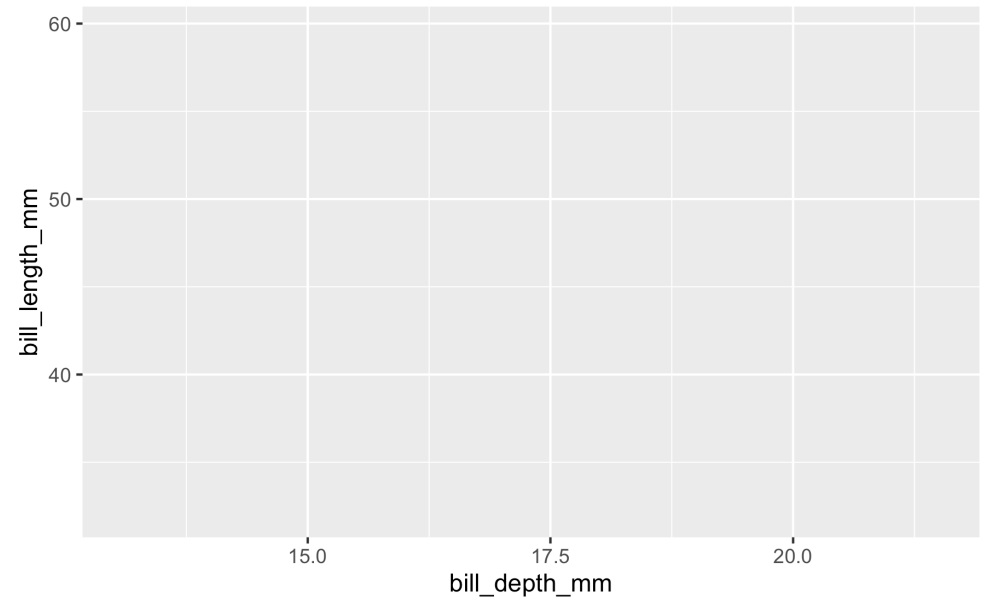
[333 rows x 8 columns]

Rows with missing values are dropped so the examples avoid ggplot2's "Removed n rows" warnings; `na.rm = TRUE` in a

Data and mapping

`ggplot()` sets the default data and aesthetic mapping for every layer that follows. With no layers there is nothing to draw, but the `x` and `y` scales already exist.

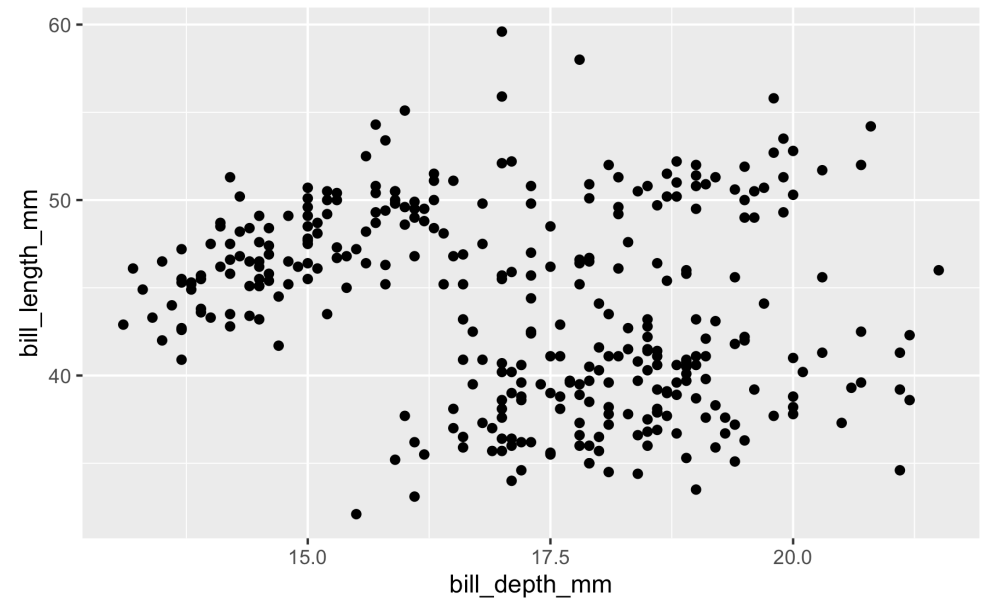
```
1 ggplot(  
2   penguins,  
3   aes(  
4     x = bill_depth_mm,  
5     y = bill_length_mm  
6   )  
7 )
```



Adding a layer

`geom_point()` adds a layer with `+`. It inherits the data and mapping from `ggplot()`, applies its default stat (`"identity"`, which leaves the data alone), and draws one point per row.

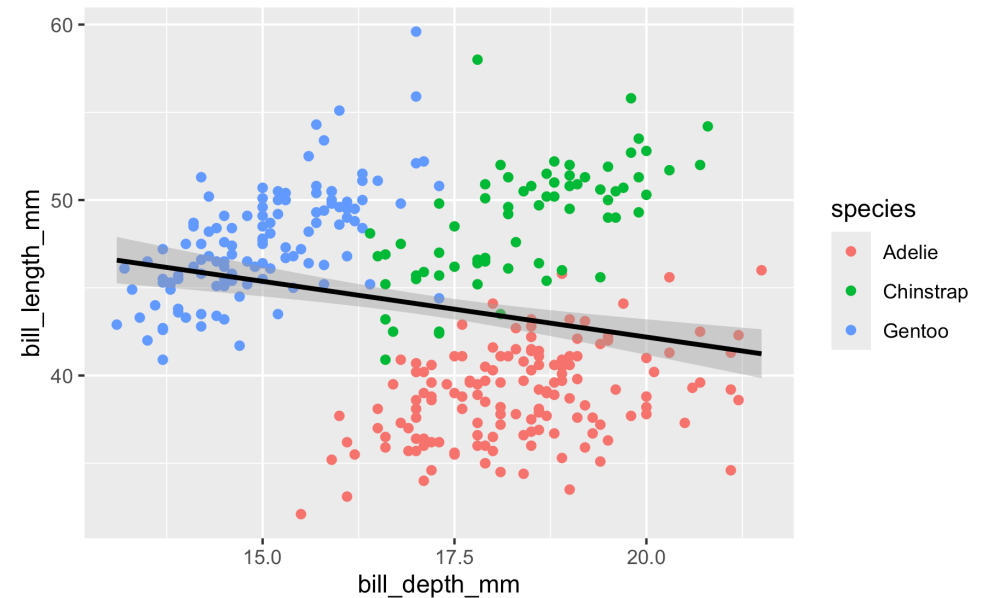
```
1 ggplot(  
2   penguins,  
3   aes(  
4     x = bill_depth_mm,  
5     y = bill_length_mm  
6   )  
7 ) +  
8   geom_point()
```



A second layer

`geom_smooth()` is a layer whose stat fits a model before drawing. The `color` mapping was given to `geom_point()` only, so the smooth layer knows nothing about species and fits one line to everything.

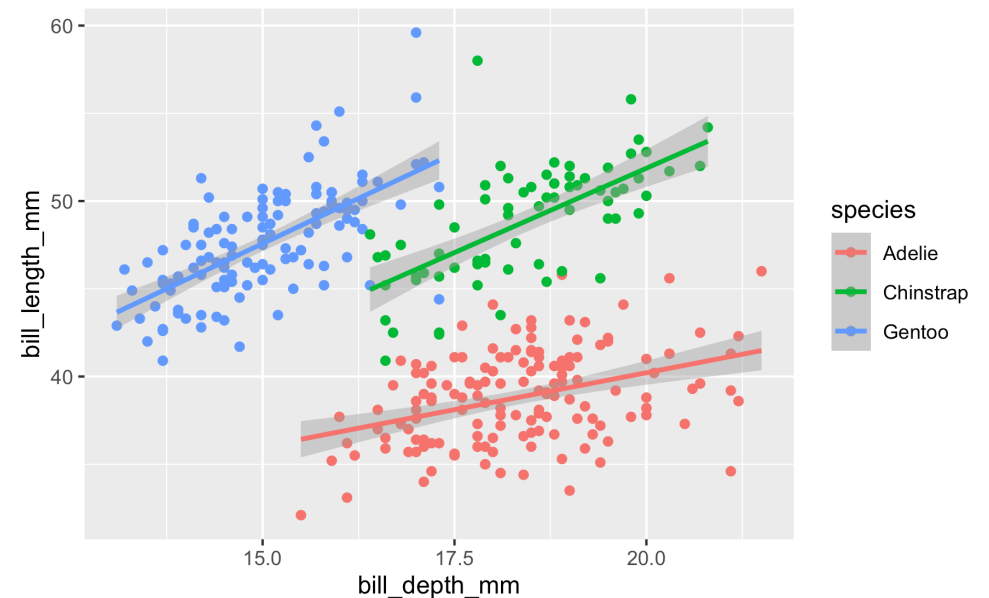
```
1 ggplot(  
2   penguins,  
3   aes(  
4     x = bill_depth_mm,  
5     y = bill_length_mm  
6   )  
7 ) +  
8   geom_point(aes(color = species)) +  
9   geom_smooth(method = "lm", color="black")
```



Inherited aesthetics

If we move `color = species` from `geom_point()` into `ggplot()`, how many regression lines will be fitted?

```
1 ggplot(  
2   penguins,  
3   aes(  
4     x = bill_depth_mm,  
5     y = bill_length_mm,  
6     color = species  
7   )  
8 ) +  
9   geom_point() +  
10  geom_smooth(method = "lm")
```

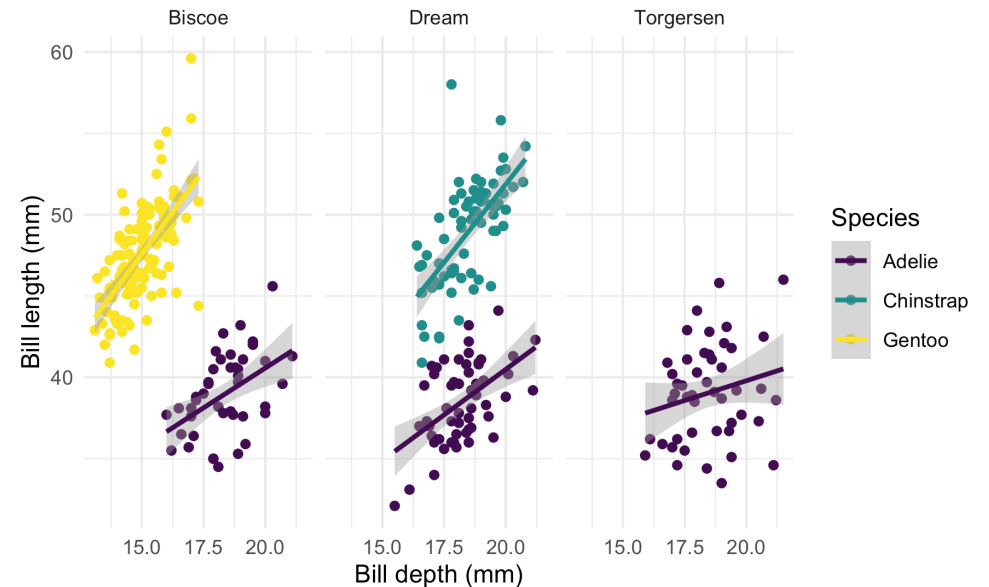


Moving the mapping into `ggplot()` makes it part of every layer. Each layer's stat is computed per group, so there is now one fit per species and the overall negative trend reverses within each (Simpson's paradox).

Other details

Scales, facets, labels, and the theme are not layers. They are also added with `+` but apply to the whole plot.

```
1 ggplot(  
2   penguins,  
3   aes(  
4     x = bill_depth_mm,  
5     y = bill_length_mm,  
6     color = species  
7   )  
8 ) +  
9   geom_point() +  
10  geom_smooth(method = "lm") +  
11  facet_wrap(~ island) +  
12  scale_color_viridis_d() +  
13  labs(  
14    x = "Bill depth (mm)",  
15    y = "Bill length (mm)",  
16    color = "Species"  
17  ) +  
18  theme_minimal()
```



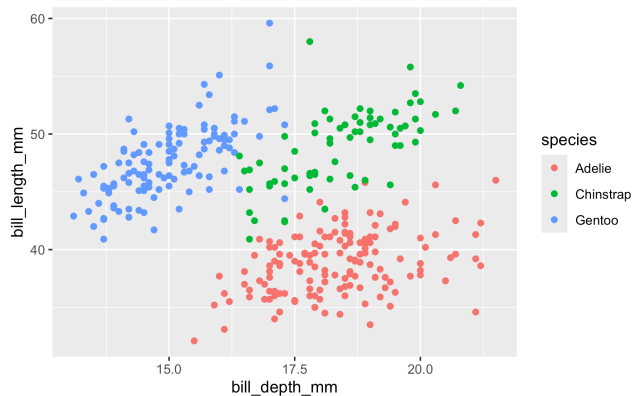
Mappings vs settings

Anything inside `aes()` is a mapping: the aesthetic varies with a variable in the data and gets a scale and a legend.

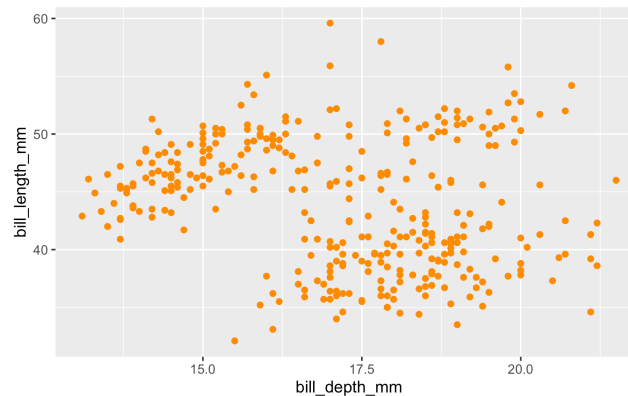
Anything outside `aes()` is a setting: a constant applied to the whole layer.

```
1 p = ggplot(penguins, aes(x = bill_depth_mm, y = bill_length_mm))
```

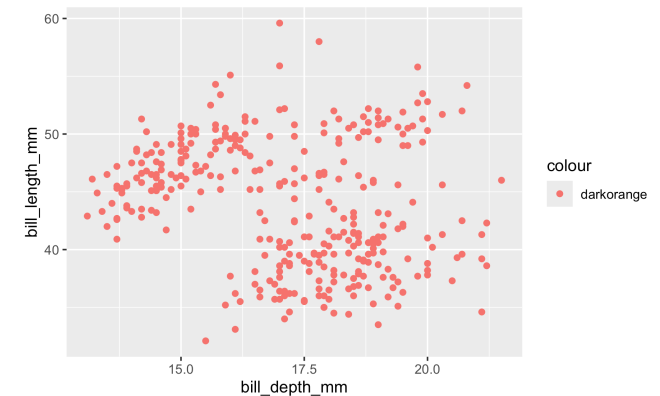
```
1 p + geom_point(  
2   aes(color = species)  
3 )
```



```
1 p + geom_point(  
2   color = "darkorange"  
3 )
```



```
1 p + geom_point(  
2   aes(color = "darkorange")  
3 )
```



The last plot is a common mistake: inside `aes()` the string is data, so it is mapped to the first color of the default palette

Data masking in aes()

`aes()` captures its arguments and evaluates them in each layer's data, the same data masking used by `dplyr`'s verbs. So a mapping can be any expression of the columns, and a column name stored in a variable needs the `.data` pronoun.

```
1 p = ggplot(penguins, aes(y = body_mass_g))
```

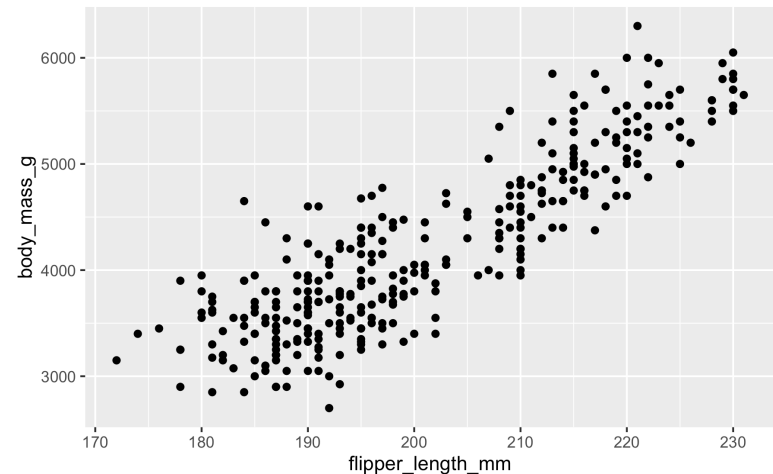
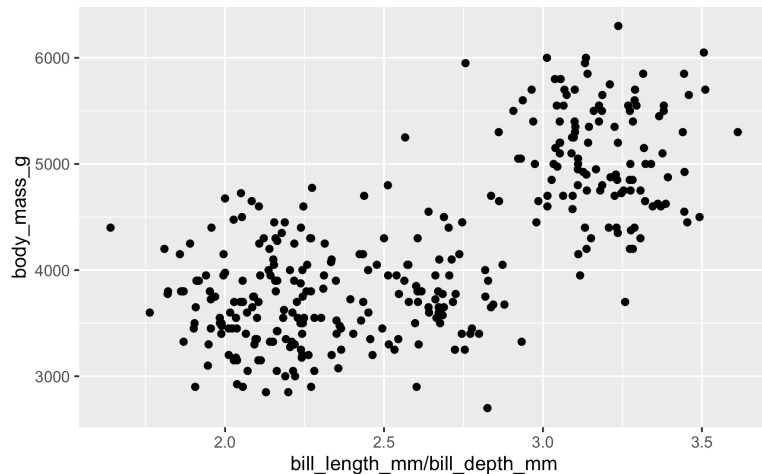
R

```
1 p + geom_point(  
2   aes(  
3     x = bill_length_mm / bill_depth_mm  
4   )  
5 )
```

R

```
1 v = "flipper_length_mm"  
2  
3 p + geom_point(  
4   aes(x = .data[[v]])  
5 )
```

R



ggplot objects

`ggplot()` returns an object describing the plot; nothing is drawn until it is printed. So a plot can be assigned, extended with `+`, stored in a list, returned from a function, or written to a file.

```
1 p = ggplot(  
2   penguins,  
3   aes(  
4     x = flipper_length_mm,  
5     y = body_mass_g  
6   )  
7 ) +  
8   geom_point()
```

```
1 class(p)[1]
```

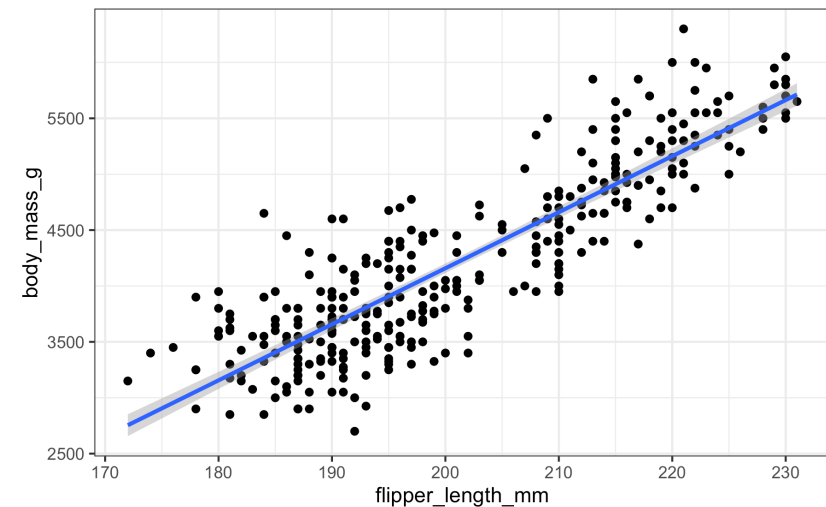
```
[1] "ggplot2::ggplot"
```

```
1 length(p@layers)
```

```
[1] 1
```

```
1 ggsave(  
2   "penguins.png", p,  
3   width = 6, height = 4  
4 )
```

```
1 p + geom_smooth(method = "lm") +  
2   theme_bw()
```



As of ggplot2 4.0, plots are S7 objects (S7 is R's newest OOP system, successor to S3 and S4), so their components are

The rest of ggplot2

We will not tour the geoms. The package [reference](#) page is organized by the components above (layers, scales, guides, facets, coordinates, themes) and is the place to find what exists and which aesthetics each one understands.

matplotlib

matplotlib & pyplot

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python.

`matplotlib.pyplot` is a collection of functions that make matplotlib work like MATLAB. Each `pyplot` function makes some change to a figure: e.g., creates a figure, creates a plotting area in a figure, plots some lines in a plotting area, decorates the plot with labels, etc.

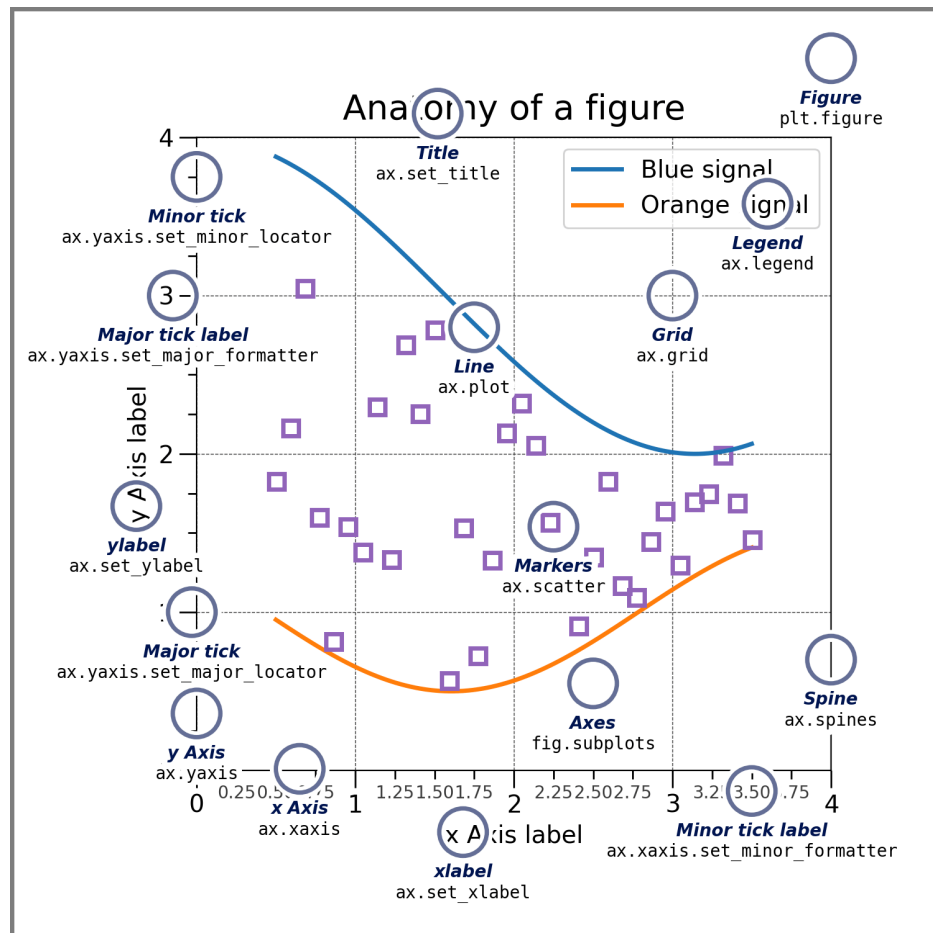
```
1 import matplotlib as mpl
2 import matplotlib.pyplot as plt
3 mpl.__version__
```

py

'3.11.2'

seaborn, plotnine, and pandas' default plotting backend draw with matplotlib, so its objects and vocabulary show up

Anatomy of a figure



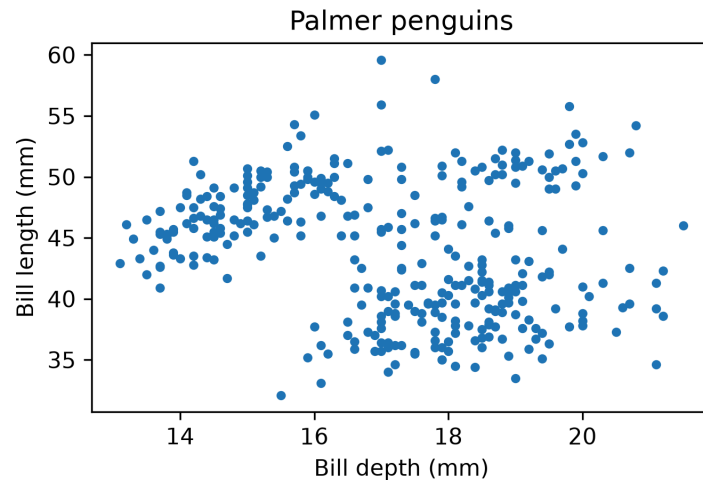
- **Figure:** the whole canvas, holding one or more Axes plus figure-wide titles and legends
- **Axes:** one plot (a panel or subplot), the data region together with its x and y Axis, title, labels, and legend. Nearly all plotting methods live here.
- **Axis:** a single number line (`ax.xaxis`, `ax.yaxis`) with its limits, ticks, and tick labels
- **Artist:** the base class for every drawn object. Figure, Axes, and Axis are Artists, and so are the lines, markers, patches, and text that plotting methods add.

Two interfaces

Implicit (pyplot):

```
1 plt.figure(figsize=(5, 3))
2 plt.scatter(
3     "bill_depth_mm", "bill_length_mm",
4     data=penguins, s=10
5 )
6 plt.xlabel("Bill depth (mm)")
7 plt.ylabel("Bill length (mm)")
8 plt.title("Palmer penguins")
```

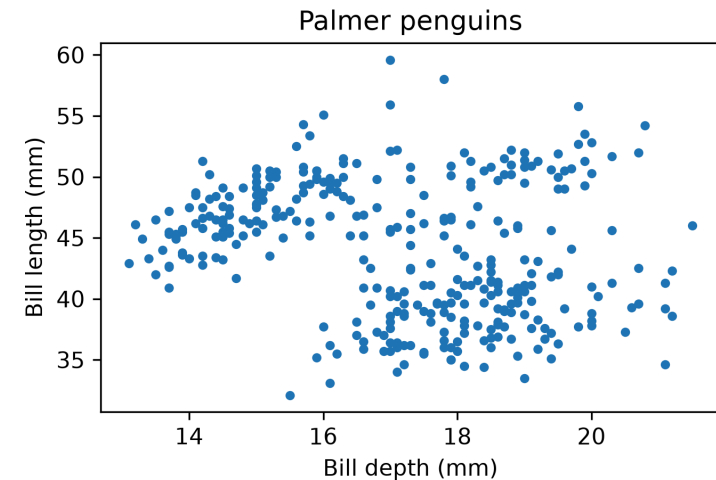
py



Explicit (object-oriented):

```
1 fig, ax = plt.subplots(figsize=(5, 3))
2 ax.scatter(
3     "bill_depth_mm", "bill_length_mm",
4     data=penguins, s=10
5 )
6 ax.set_xlabel("Bill depth (mm)")
7 ax.set_ylabel("Bill length (mm)")
8 ax.set_title("Palmer penguins")
```

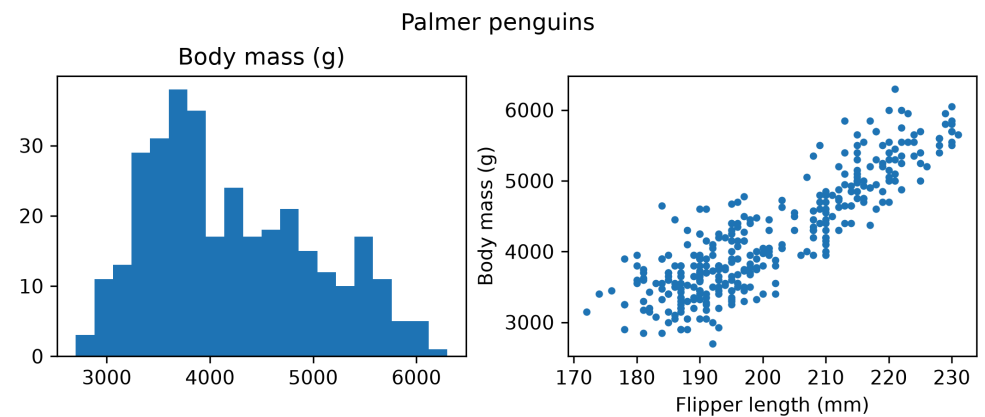
py



Subplots (00)

`plt.subplots(nrows, ncols)` returns a Figure and its Axes. Each panel is drawn and labeled through its own object; the figure-wide title belongs to the Figure.

```
1 fig, axs = plt.subplots(
2     1, 2, figsize=(7, 3),
3     layout="constrained"
4 )
5 axs[0].hist(
6     penguins["body_mass_g"], bins=20
7 )
8 axs[0].set_title("Body mass (g)")
9
10 axs[1].scatter(
11     "flipper_length_mm", "body_mass_g",
12     data=penguins, s=8
13 )
14 axs[1].set(
15     xlabel="Flipper length (mm)",
16     ylabel="Body mass (g)"
17 )
18 fig.suptitle("Palmer penguins")
```

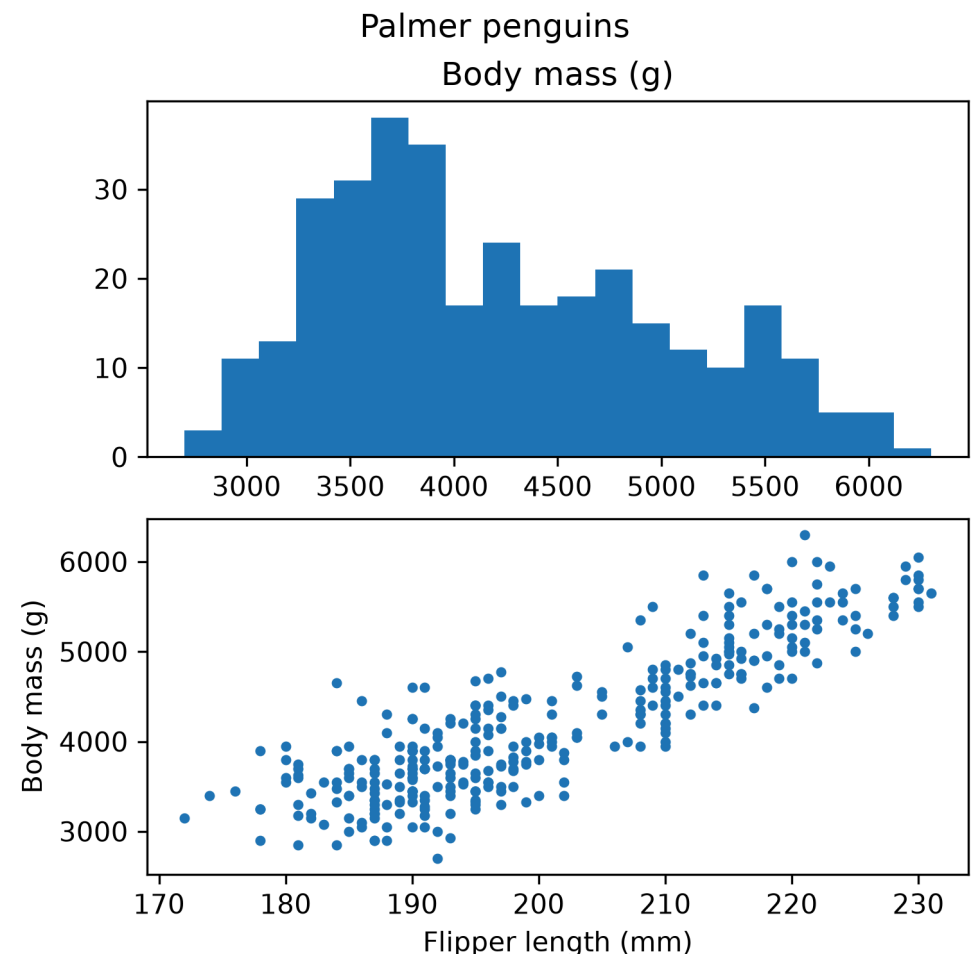


`axs` is a NumPy array of `Axes` objects, 1D for a single row or column and 2D for a grid. With one panel a single `Axes` is

Subplots (implicit)

`plt.subplot(nrows, ncols, index)` adds the panel at `index` (counting from 1, by row) and makes it the current Axes; subsequent `plt.*` calls draw into whichever panel was selected last.

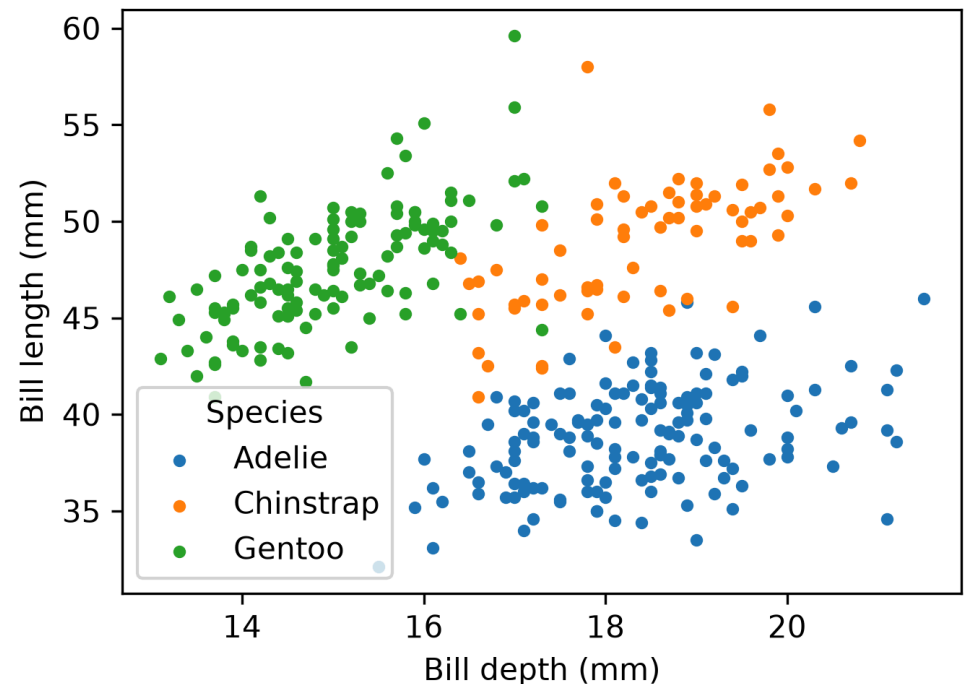
```
1 plt.figure(  
2     figsize=(5, 5), layout="constrained"  
3 )  
4 plt.subplot(2, 1, 1)  
5 plt.hist(  
6     penguins["body_mass_g"], bins=20  
7 )  
8 plt.title("Body mass (g)")  
9  
10 plt.subplot(2, 1, 2)  
11 plt.scatter(  
12     "flipper_length_mm", "body_mass_g",  
13     data=penguins, s=8  
14 )  
15 plt.xlabel("Flipper length (mm)")  
16 plt.ylabel("Body mass (g)")  
17 plt.suptitle("Palmer penguins")
```



Grouping by hand

matplotlib draws what it is told: `ax.scatter()` takes arrays and either a single color or numeric values run through a colormap (`c=`, `cmap=`). There is no notion of mapping a categorical variable to color, so coloring by species means splitting the data, drawing each subset with a `label`, and asking for a legend. Every additional variable (shape by sex, one panel per island) is another loop.

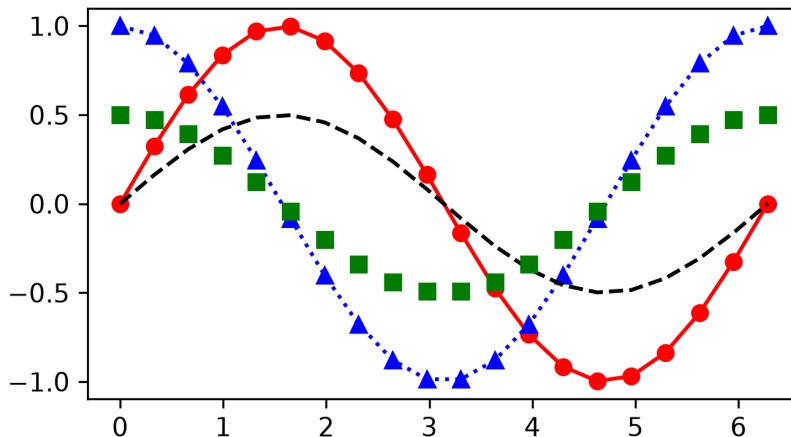
```
1 fig, ax = plt.subplots(figsize=(5, 3.5))
2 for s, d in penguins.groupby("species"):
3     ax.scatter(
4         "bill_depth_mm", "bill_length_mm",
5         data=d, label=s, s=10
6     )
7 ax.set_xlabel("Bill depth (mm)")
8 ax.set_ylabel("Bill length (mm)")
9 ax.legend(title="Species")
```



Format strings

`ax.plot()` accepts a format string, "`[marker][line][color]`", as shorthand for the `marker`, `linestyle`, and `color` keywords. Each part is optional.

```
1 import numpy as np
2 x = np.linspace(0, 2 * np.pi, 20)
3
4 fig, ax = plt.subplots(figsize=(5, 2.75))
5 ax.plot(x, np.sin(x), "o-r")
6 ax.plot(x, np.cos(x), "^:b")
7 ax.plot(x, np.sin(x) / 2, "--k")
8 ax.plot(x, np.cos(x) / 2, "sg")
```



marker	line	color
.	point	solid
o	circle	dashed
s	square	dash-dot
D	diamond	dotted
^	triangle up	
v	triangle down	
+	plus	
x	cross	
*	star	
r		red
g		green
b		blue
c		cyan
m		magenta
y		yellow
k		black
w		white
C0-C9		cycle

seaborn

seaborn

Seaborn is a library for making statistical graphics in Python. It builds on top of matplotlib and integrates closely with pandas data structures. ... Its plotting functions operate on dataframes and arrays containing whole datasets and internally perform the necessary semantic mapping and statistical aggregation to produce informative plots. Its dataset-oriented, declarative API lets you focus on what the different elements of your plots mean, rather than on the details of how to draw them.

```
1 import seaborn as sns
2 sns.__version__
```

py

```
'0.13.2'
```

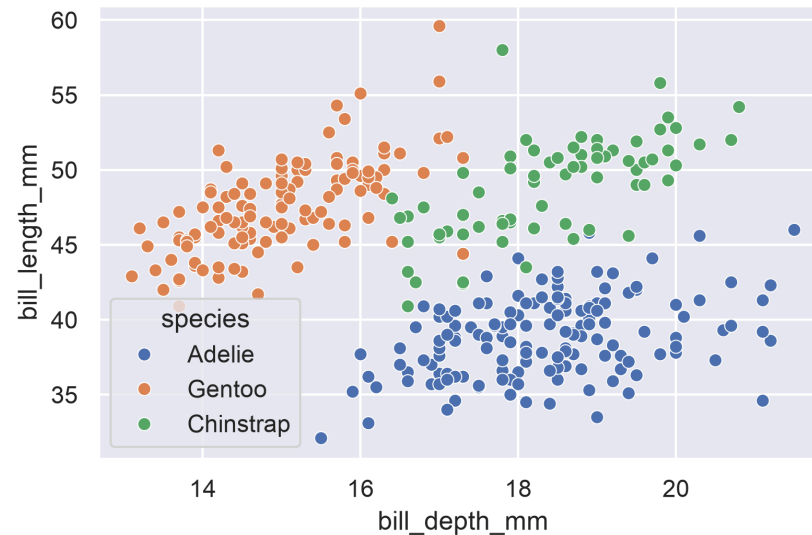
```
1 sns.set_theme()
```

py

Semantic mappings

`hue` is seaborn's name for a color mapping, the equivalent of ggplot2's `color` aesthetic. Python has no data masking, so variables are strings naming columns in `data`.

```
1 sns.scatterplot(  
2     data=penguins,  
3     x="bill_depth_mm",  
4     y="bill_length_mm",  
5     hue="species"  
6 )
```

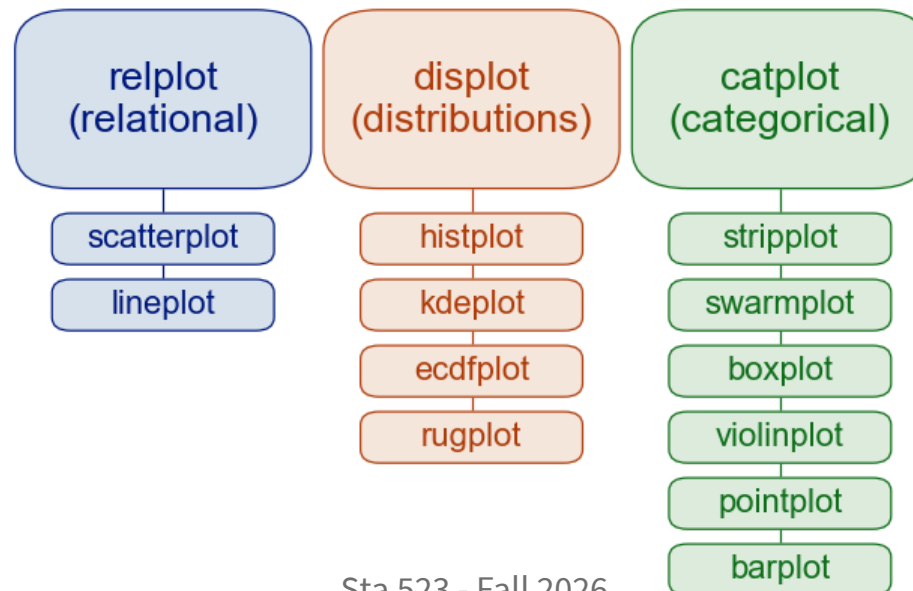


`data` can be a pandas DataFrame, a dict of arrays or lists, or another DataFrame such as polars (converted to pandas).

Figure-level vs axes-level functions

seaborn's plotting functions come in two flavors:

- axes-level functions (`scatterplot()`, `histplot()`, `boxplot()`, ...) draw one plot on a matplotlib `Axes` (the current one, or one passed as `ax=`) and return that `Axes`, so they mix freely with matplotlib code.
- figure-level functions (`relplot()`, `displot()`, `catplot()`) create their own `Figure`, axes-level function are chosen via `kind=`, faceting via `col=` and `row=`, with a legend placed outside, and return a `FacetGrid`.

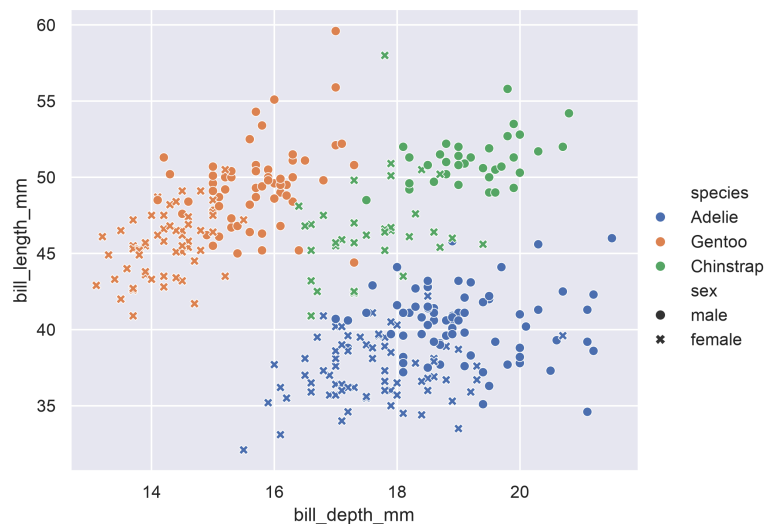


relplot()

`relplot()` shows the relationship between two numeric variables, with `kind="scatter"` (the default) or `kind="line"`. A line plot aggregates rows that share an `x` value, drawing their mean and a 95% bootstrap CI.

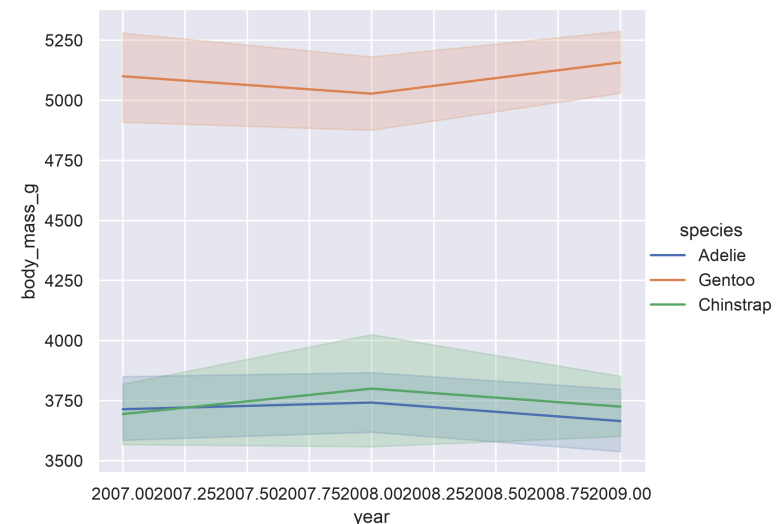
```
1 g = sns.relplot(  
2     data=penguins, x="bill_depth_mm",  
3     y="bill_length_mm", hue="species",  
4     style="sex", height=5, aspect=1.2  
5 )
```

py



```
1 g = sns.relplot(  
2     data=penguins, x="year",  
3     y="body_mass_g", hue="species",  
4     kind="line", height=5, aspect=1.2  
5 )
```

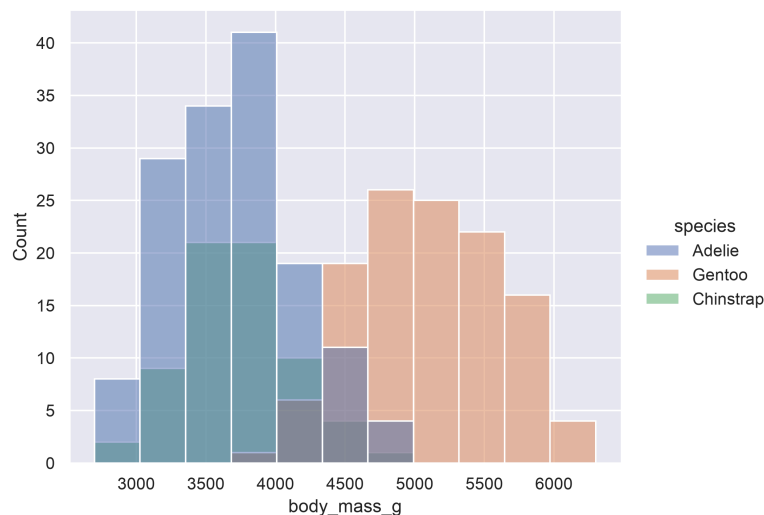
py



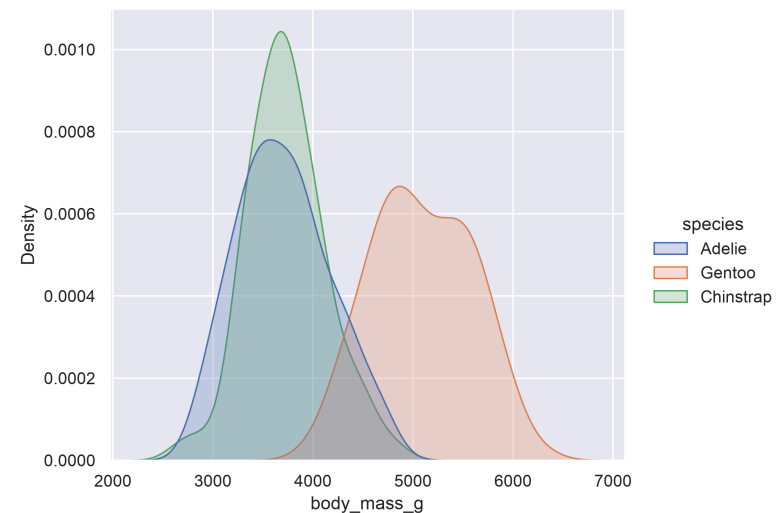
displot()

`displot()` shows the distribution of a numeric variable, with `kind="hist"` (the default), `"kde"`, or `"ecdf"`.

```
1 g = sns.displot(  
2     data=penguins, x="body_mass_g",  
3     hue="species",  
4     height=5, aspect=1.2  
5 )
```



```
1 g = sns.displot(  
2     data=penguins, x="body_mass_g",  
3     hue="species", kind="kde", fill=True,  
4     common_norm=False,  
5     height=5, aspect=1.2  
6 )
```

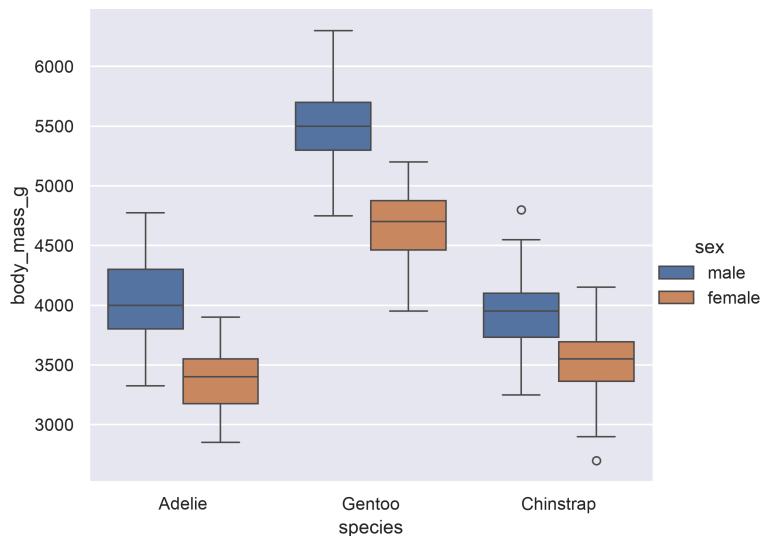


With `hue`, seaborn defaults to scaling density areas by group size. Use `common_norm=False` to normalize each group

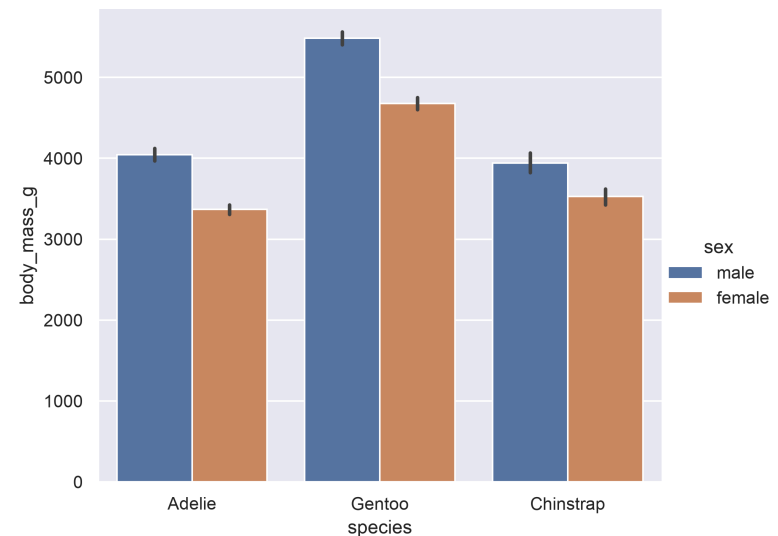
catplot()

`catplot()` shows data grouped by the levels of a categorical variable, with `kind="strip"` (the default), `"box"`, `"violin"`, `"bar"`, and others.

```
1 g = sns.catplot(  
2     data=penguins, x="species",  
3     y="body_mass_g", hue="sex",  
4     kind="box", height=5, aspect=1.2  
5 )
```



```
1 g = sns.catplot(  
2     data=penguins, x="species",  
3     y="body_mass_g", hue="sex",  
4     kind="bar", height=5, aspect=1.2  
5 )
```



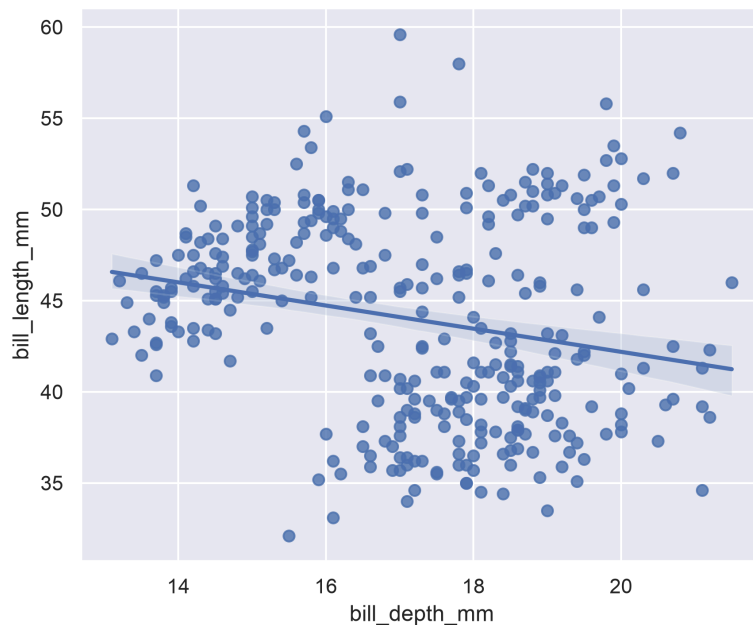
Bars can represent different quantities: `geom_bar()` and `catplot(kind="count")` count observations; `catplot(kind="bar")` estimates means by default, with a 95% bootstrap CI as the error bar; `geom_col()` draws values

lmpplot()

`lmpplot()` draws a scatterplot with a fitted regression line and its 95% confidence band, the equivalent of `geom_point()` + `geom_smooth(method = "lm")`. With `hue` there is one fit per group.

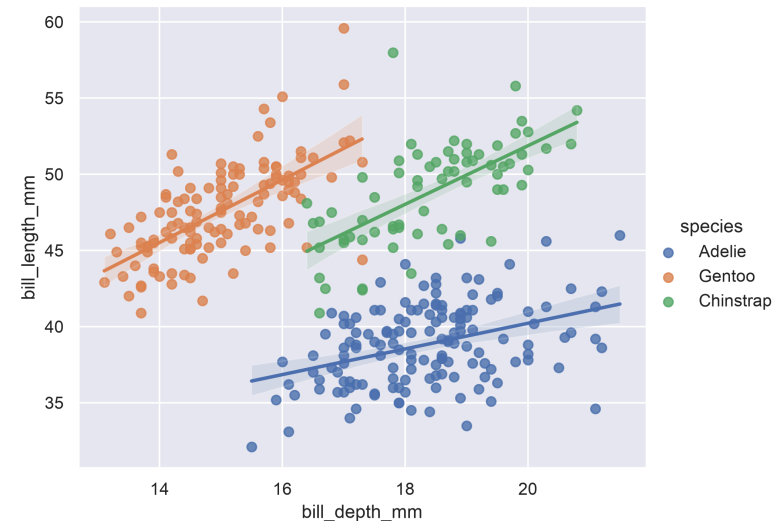
```
1 g = sns.lmplot(  
2   data=penguins, x="bill_depth_mm",  
3   y="bill_length_mm",  
4   height=5, aspect=1.2  
5 )
```

py



```
1 g = sns.lmplot(  
2   data=penguins, x="bill_depth_mm",  
3   y="bill_length_mm", hue="species",  
4   height=5, aspect=1.2  
5 )
```

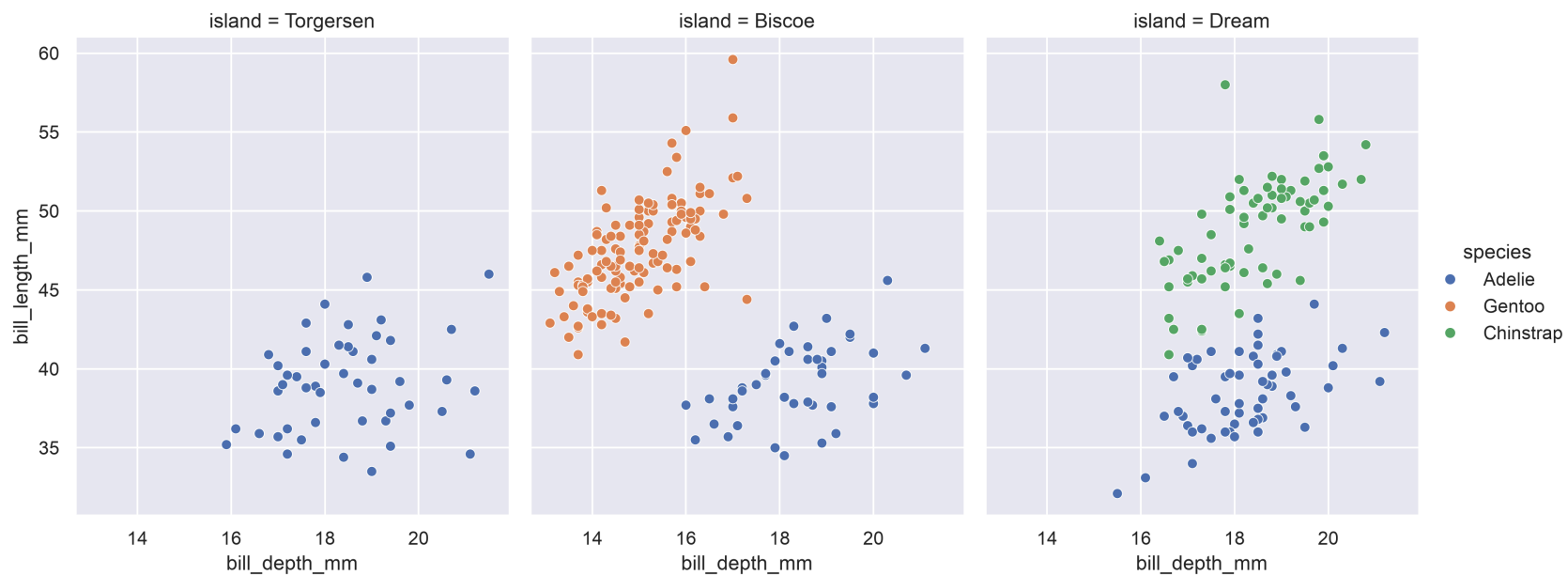
py



Faceting

Figure-level functions facet with `col=`, `row=`, and `col_wrap=`. Panel size is set by `height` and `aspect`, not `figsize`.

```
1 g = sns.relplot(  
2     data=penguins, x="bill_depth_mm", y="bill_length_mm",  
3     hue="species", col="island",  
4     height=5, aspect=0.8  
5 )
```

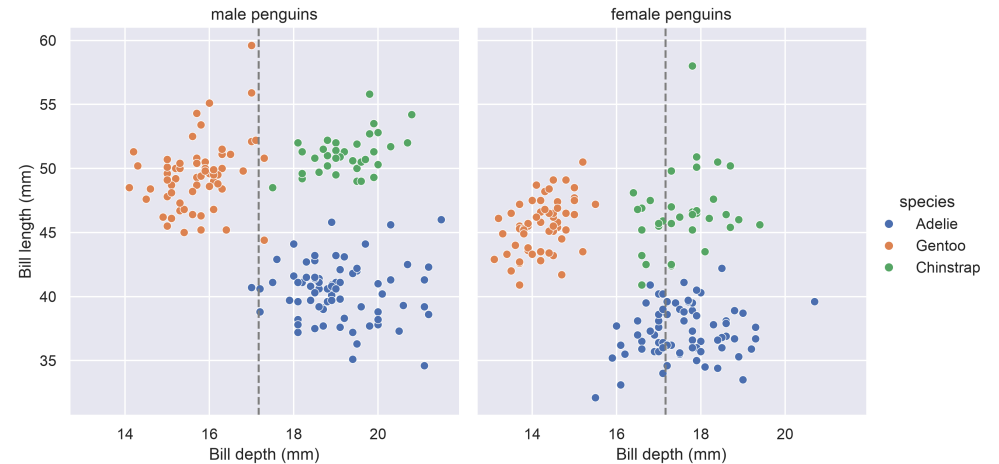


Customizing a FacetGrid

`FacetGrid` methods cover common tweaks and chain; anything else goes through the wrapped matplotlib objects (`g.axes`, `g.figure`).

```
1 g = sns.relplot(  
2     data=penguins, x="bill_depth_mm",  
3     y="bill_length_mm", hue="species",  
4     col="sex", height=5, aspect=0.9  
5 ).set_axis_labels(  
6     "Bill depth (mm)", "Bill length (mm)"  
7 ).set_titles(  
8     "{col_name} penguins"  
9 )  
10  
11 for ax in g.axes.flat:  
12     ax.axvline(  
13         penguins["bill_depth_mm"].mean(),  
14         color="grey", ls="--"  
15     )
```

py

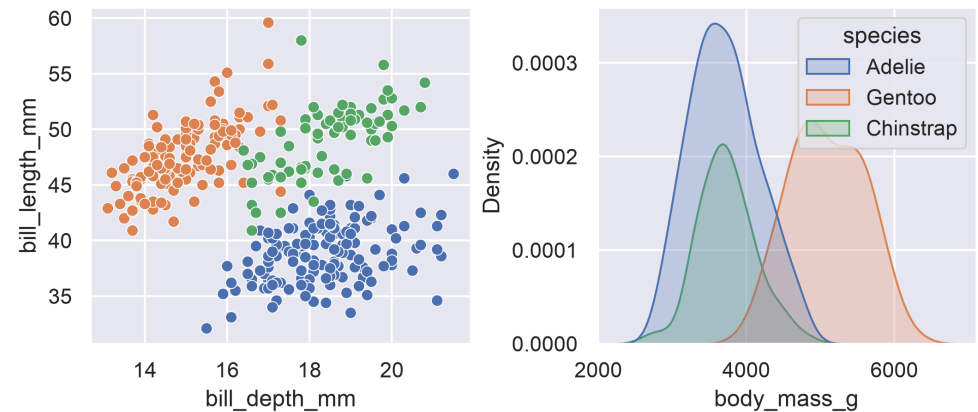


Axes-level functions

You already have `fig, axs = plt.subplots(1, 2)`. Which function can draw into `axs[0]`: `sns.scatterplot()` or `sns.relplot()`?

Because axes-level functions take an `ax=` argument, they can draw into any figure: several can share a `plt.subplots()` grid alongside plain matplotlib content. Their legend is placed inside the axes.

```
1 fig, axs = plt.subplots(
2     1, 2, figsize=(7, 3),
3     layout="constrained"
4 )
5 sns.scatterplot(
6     data=penguins, x="bill_depth_mm",
7     y="bill_length_mm", hue="species",
8     ax=axs[0]
9 )
10 sns.kdeplot(
11     data=penguins, x="body_mass_g",
12     hue="species", fill=True,
13     ax=axs[1]
14 )
15 axs[0].get_legend().remove()
```



Layering

Calls to axes-level functions on the same `Axes` stack, so layers are built by calling functions in order (each draws immediately) rather than by adding them to a plot object. Each layer with `hue` would replace the legend with its own, so later layers pass `legend=False`.

```
1 fig, ax = plt.subplots(figsize=(5, 3.5)) py
2 sns.kdeplot(
3     data=penguins, x="bill_depth_mm",
4     y="bill_length_mm", hue="species",
5     ax=ax
6 )
7 sns.scatterplot(
8     data=penguins, x="bill_depth_mm",
9     y="bill_length_mm", hue="species",
10    alpha=0.5, legend=False, ax=ax
11 )
12 sns.rugplot(
13     data=penguins, x="bill_depth_mm",
14     y="bill_length_mm", hue="species",
15     legend=False, ax=ax
16 )
```

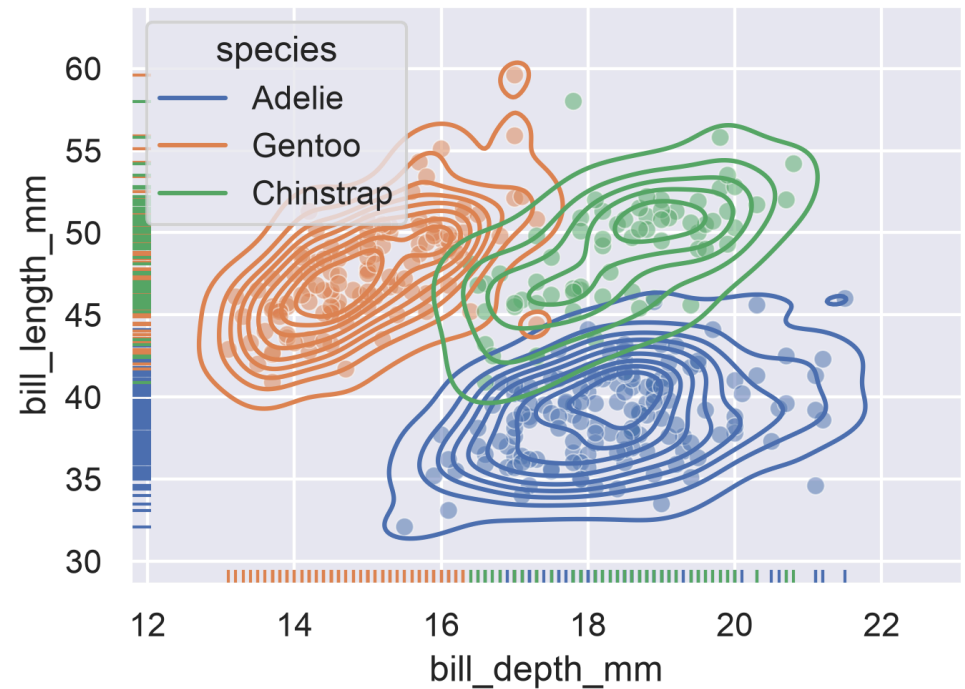


Figure-level or axes-level?

Figure-level functions

- facet by data variables with `col=` and `row=`
- legend outside the plot by default
- `kind=` switches the representation without changing the call
- sized per facet with `height` and `aspect`
- many options are not in the signature but pass through as `**kwargs`

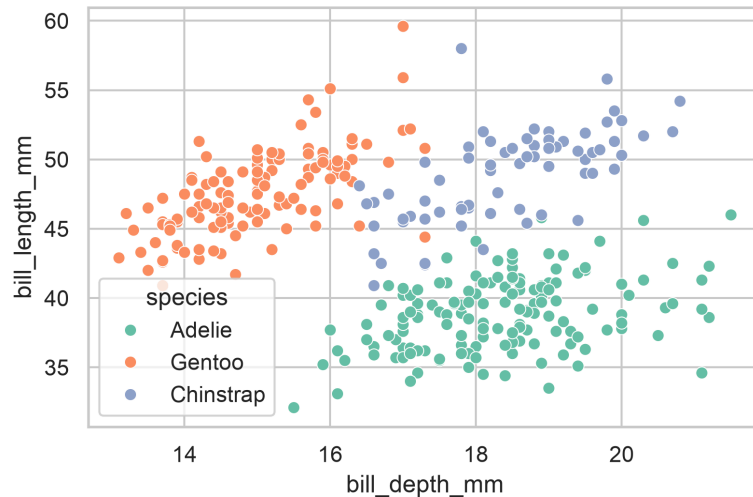
Axes-level functions

- drop into any matplotlib figure with `ax=`, so they compose with subplots and plain matplotlib
- plot-specific parameters appear in the signature; additional matplotlib options pass through `**kwargs`
- sized like any matplotlib figure with `figsize`
- the choice when the plot is one panel of something larger

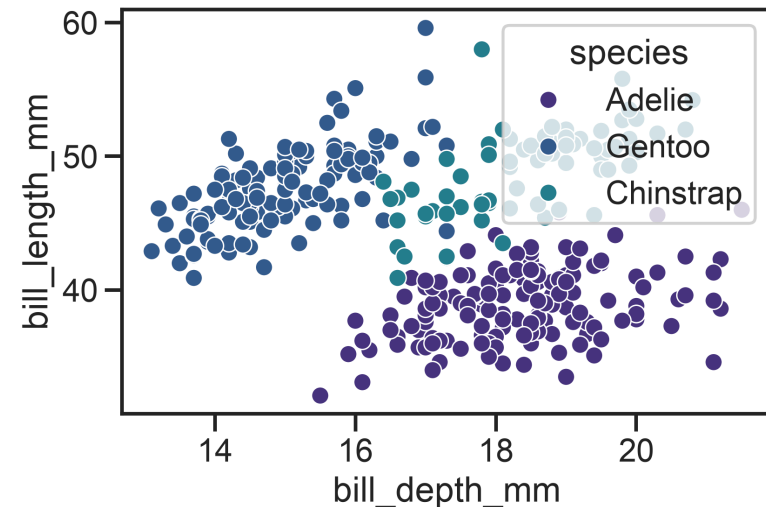
Themes and palettes

`sns.set_theme()` sets the `style` (`darkgrid`, `whitegrid`, `dark`, `white`, `ticks`), the `context` (`paper`, `notebook`, `talk`, `poster`, which scale fonts and line widths), and the default `palette` for every subsequent figure. `palette=` on any function applies to one plot and `with sns.axes_style("white"):` to a block.

```
1 sns.set_theme(  
2     style="whitegrid", palette="Set2"  
3 )  
4 sns.scatterplot(  
5     data=penguins, x="bill_depth_mm",  
6     y="bill_length_mm", hue="species"  
7 )
```



```
1 sns.set_theme(  
2     style="ticks", context="talk", palette="v"  
3 )  
4 sns.scatterplot(  
5     data=penguins, x="bill_depth_mm",  
6     y="bill_length_mm", hue="species"  
7 )
```



seaborn.objects

seaborn.objects

The `seaborn.objects` namespace was introduced in version 0.12 as a completely new interface for making seaborn plots. ... the new interface aims to support end-to-end plot specification and customization without dropping down to matplotlib (although it will remain possible to do so if necessary).

```
1 import seaborn.objects as so
```

py

This is seaborn's take on the grammar of graphics.

A `Plot` holds the data and mappings, each `.add()` is a layer combining a `Mark` (`Dot`, `Line`, `Bar`, `Area`, ...) with an optional `Stat` (`Agg`, `Hist`, `PolyFit`, ...) and `Move` (`Dodge`, `Jitter`, `Stack`, ...), and `.facet()`, `.scale()`, `.label()`, and `.theme()` apply to the whole plot.

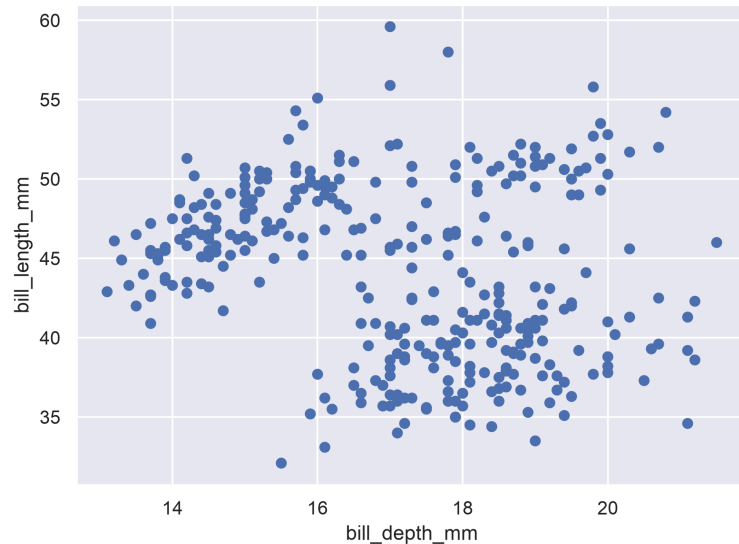
A `Plot` is a specification that is rendered by `.show()`, `.plot()`, or `.save()`, or automatically when displayed in a Jupyter notebook.

The objects interface is still marked as under development and covers less than ggplot2 (no polar coordinates, fewer

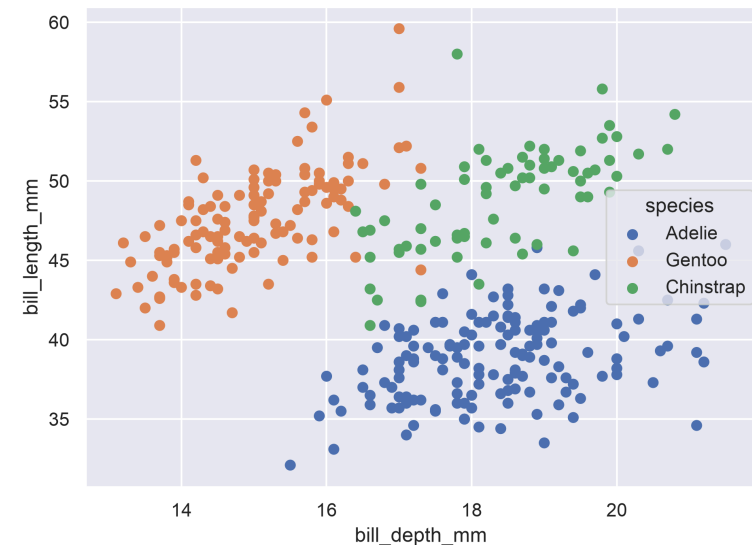
Building a plot

`so.Plot()` takes the data and the mappings; `.add()` adds a layer, and a mark with no stat draws the data as is.

```
1 ( so.Plot(  
2     penguins,  
3     x="bill_depth_mm",  
4     y="bill_length_mm"  
5 )  
6 .add(so.Dot())  
7 ).show()
```



```
1 ( so.Plot(  
2     penguins,  
3     x="bill_depth_mm",  
4     y="bill_length_mm",  
5     color="species"  
6 )  
7 .add(so.Dot())  
8 ).show()
```



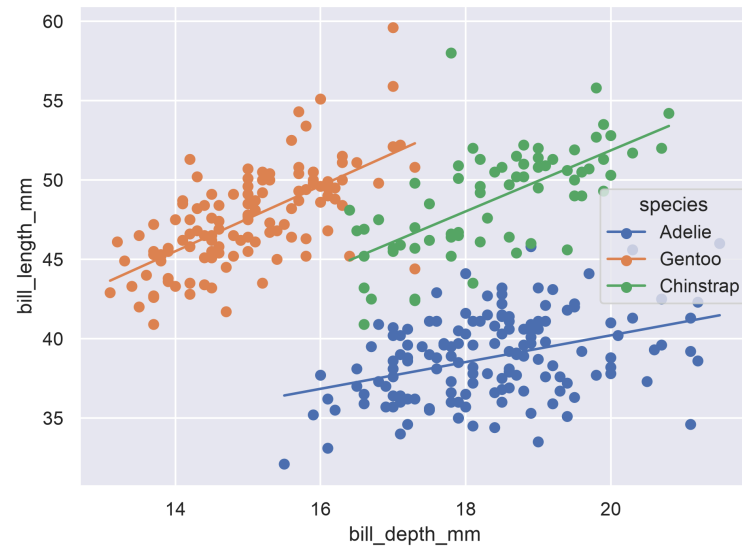
Layers and stats

Each `.add()` is a layer; a `Stat` passed with the mark transforms the data before it is drawn, per group when a mapping such as `color` is present.

```

1 ( so.Plot(
2     penguins,
3     x="bill_depth_mm",
4     y="bill_length_mm",
5     color="species"
6 )
7 .add(so.Dot())
8 .add(so.Line(), so.PolyFit(order=1))
9 ).show()

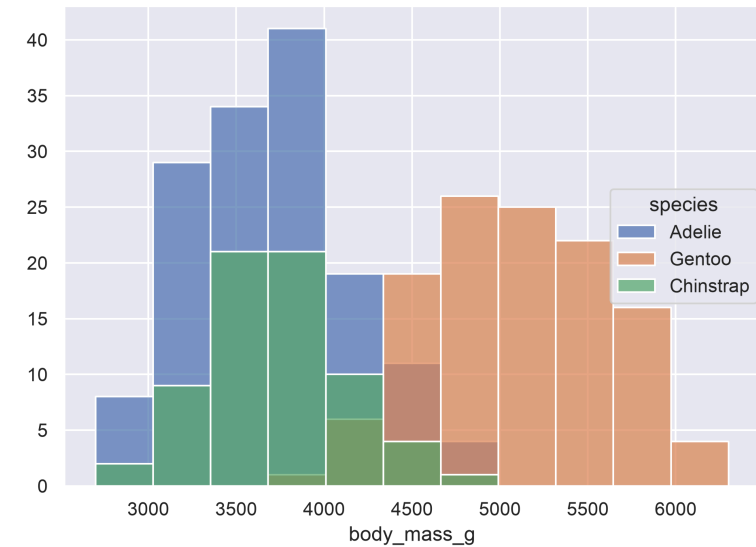
```



```

1 ( so.Plot(
2     penguins,
3     x="body_mass_g", color="species"
4 )
5 .add(so.Bars(), so.Hist())
6 ).show()

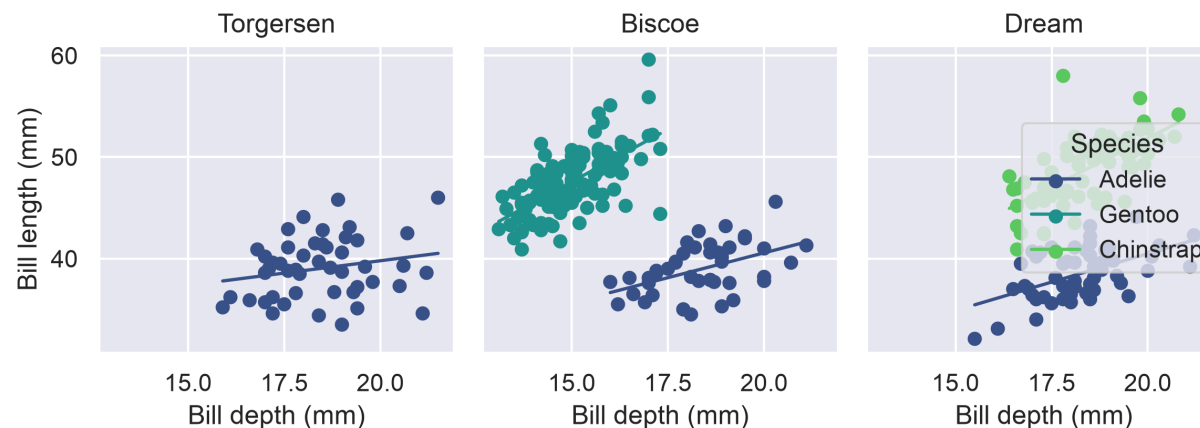
```



Facets, scales, and labels

The remaining methods are plot-wide, mirroring the last step of the ggplot2 example.

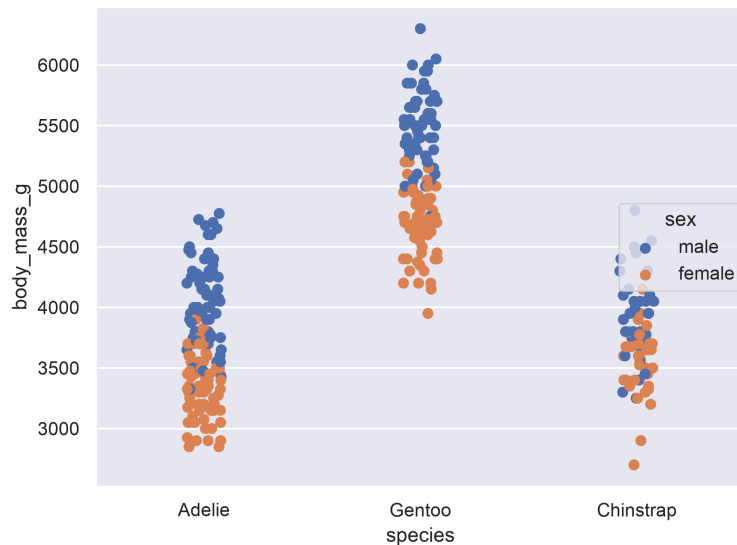
```
1 ( so.Plot(  
2     penguins, x="bill_depth_mm", y="bill_length_mm", color="species"  
3 )  
4 .add(so.Dot())  
5 .add(so.Line(), so.PolyFit(order=1))  
6 .facet(col="island")  
7 .scale(color="viridis")  
8 .label(x="Bill depth (mm)", y="Bill length (mm)", color="Species")  
9 .layout(size=(8, 3))  
10 ).show()
```



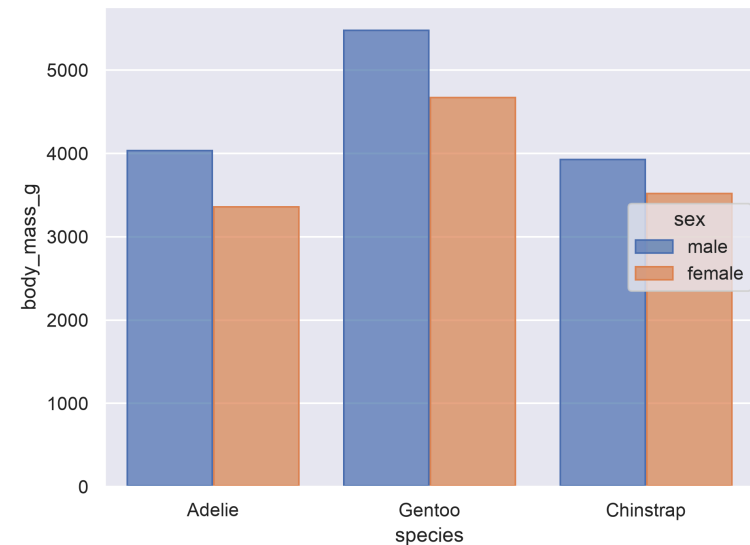
Moves

A **Move** adjusts positions after the stat: **Dodge()** places groups side by side, **Jitter()** spreads overlapping points, **Stack()** piles them up.

```
1 ( so.Plot(  
2     penguins,  
3     x="species", y="body_mass_g",  
4     color="sex"  
5 )  
6 .add(so.Dot(), so.Jitter())  
7 ).show()
```



```
1 ( so.Plot(  
2     penguins,  
3     x="species", y="body_mass_g",  
4     color="sex"  
5 )  
6 .add(so.Bar(), so.Aggregate(), so.Dodge())  
7 ).show()
```



ggplot2 to seaborn.objects

ggplot2

```
ggplot(data, aes(x, y, color))  
+ geom_point(),+ geom_line()  
+ geom_bar()  
+ geom_histogram()  
+ geom_smooth(method = "lm", se = FALSE)  
+ stat_summary(fun = mean, geom = "point")  
position = "dodge","jitter","stack"  
+ facet_wrap(~ v),+ facet_grid(r ~ c)  
+ scale_color_viridis_d()  
+ labs()  
+ coord_cartesian(xlim = )  
+ theme()  
ggsave()  
printing the object
```

seaborn.objects

```
so.Plot(data, x=, y=, color=)  
.add(so.Dot()),.add(so.Line())  
.add(so.Bar(), so.Count())  
.add(so.Bars(), so.Hist())  
.add(so.Line(), so.PolyFit(order=1))  
.add(so.Dot(), so.Agg())  
so.Dodge(),so.Jitter(),so.Stack()  
.facet(col="v", wrap=),.facet(row=, col=)  
.scale(color="viridis")  
.label()  
.limit(x=)  
.theme({...}) with matplotlib rcParams  
.save()  
.show()
```

Summary

Takeaways

- A plot is data, mappings from variables to aesthetics, and layers (geom + stat + position) drawn on shared scales, facets, and coordinates under a theme. `ggplot2` is this grammar; `seaborn.objects` is its closest Python analog.
- `matplotlib` is the drawing layer beneath `seaborn`, `plotnine`, and `pandas`' default plotting backend. Know `Figure`, `Axes`, and `Axis`, the `plt.f()` to `ax.set_f()` naming, and prefer the object-oriented style.
- `seaborn`'s figure-level functions create and manage a figure; axes-level functions draw on an `Axes` supplied through `ax=.relplot()`, `displot()`, and `catplot()` select plot types with `kind=`; `lmplot()` provides regression plots. All four support faceting and return a `FacetGrid`. `x`, `y`, `hue`, `size`, and `style` are aesthetic mappings; keywords such as `color=` are settings.
- Use whichever gets to the plot in the fewest concepts, and drop down to `matplotlib` for the last details.

Learning more

ggplot2

- Reference
- ggplot2 book
- R4DS - Visualize
- Extension gallery

matplotlib

- Quick start guide
- Plot types
- Cheatsheets

seaborn & plotnine

- User guide
- API reference
- Example gallery
- The objects interface
- plotnine, ggplot2 in Python

Reference

Naming conventions

Every `plt.*` function acts on the current Figure or Axes; the object-oriented equivalents are methods of `Figure` or `Axes`, with setters named `set_*` (and matching `get_*`).

pyplot (current figure / axes)

```
plt.figure(), plt.subplots()
```

```
plt.plot(), plt.scatter(), plt.hist(), plt.bar()
```

```
plt.title()
```

```
plt.xlabel(), plt.ylabel()
```

```
plt.xlim(), plt.ylim()
```

```
plt.xscale(), plt.yscale()
```

```
plt.xticks(), plt.yticks()
```

```
plt.legend(), plt.grid()
```

```
plt.suptitle(), plt.savefig()
```

```
plt.gcf(), plt.gca()
```

object-oriented

```
fig = plt.figure(), fig, ax = plt.subplots()
```

```
ax.plot(), ax.scatter(), ax.hist(), ax.bar()
```

```
ax.set_title()
```

```
ax.set_xlabel(), ax.set_ylabel()
```

```
ax.set_xlim(), ax.set_ylim()
```

```
ax.set_xscale(), ax.set_yscale()
```

```
ax.set_xticks(), ax.set_xticklabels(), ...
```

```
ax.legend(), ax.grid()
```

```
fig.suptitle(), fig.savefig()
```

```
fig, ax
```

The FacetGrid

The figure-level functions covered here return a `FacetGrid`, a wrapper around the matplotlib `Figure` and its `Axes` that also manages the legend and the facet titles.

```
1 type(g)
```

<class 'seaborn.axisgrid.FacetGrid'>

Attribute	Description	Method	Description
<code>figure</code>	the matplotlib <code>Figure</code>	<code>set_axis_labels()</code>	x and y labels on the outer facets
<code>ax</code>	the <code>Axes</code> , when there is a single facet	<code>set_titles()</code>	facet titles from a template, e.g. " <code>{col_name}</code> "
<code>axes</code>	array of <code>Axes</code> , one per facet	<code>set()</code>	call <code>Axes.set()</code> on every facet
<code>axes_dict</code>	facet name(s) to <code>Axes</code>	<code>refline()</code>	add reference lines to every facet
<code>legend</code>	the <code>Legend</code> , if there is one	<code>map()</code>	apply a plotting function to every facet
		<code>savefig()</code>	save the figure

Vocabulary

Concept	ggplot2	matplotlib	seaborn	seaborn.objects
data	<code>ggplot(data)</code>	arrays or <code>data=</code>	<code>data=</code>	<code>so.Plot(data)</code>
mapping	<code>aes(color = v)</code>	a loop over groups	<code>hue="v", size=, style=</code>	<code>color="v", pointsize=</code>
setting	<code>color = "red"</code>	<code>color="red"</code>	<code>color="red"</code>	<code>so.Dot(color="red")</code>
geom	<code>geom_*</code>	<code>ax.scatter()</code> , <code>ax.plot()</code> , ...	<code>kind=</code> or an axes-level function	<code>so.Dot()</code> , <code>so.Bar()</code> , ...
stat	<code>stat_*</code>	by hand	built into each function	<code>so.Hist()</code> , <code>so.Agg()</code> , ...
position	<code>position =</code>	by hand	<code>dodge=, multiple=</code>	<code>so.Dodge()</code> , <code>so.Stack()</code>
facets	<code>facet_wrap()</code>	<code>plt.subplots()</code>	<code>col=, row=</code>	<code>.facet()</code>
scales	<code>scale_*</code>	<code>ax.set_xscale()</code> , <code>colormaps</code>	<code>palette=, hue_norm=</code>	<code>.scale()</code>
labels	<code>labs()</code>	<code>ax.set_xlabel()</code> , <code>ax.set_title()</code>	<code>g.set_axis_labels()</code>	<code>.label()</code>
theme	<code>theme_*</code> (), <code>theme()</code>	<code>rcParams</code> , style sheets	<code>sns.set_theme()</code>	<code>.theme()</code>
object	<code>ggplot</code>	Figure, Axes	FacetGrid or Axes	Plot
save	<code>ggsave()</code>	<code>fig.savefig()</code>	<code>g.savefig()</code> , <code>fig.savefig()</code>	<code>.save()</code>